QCompile

Academic Project Report

A small compiler and classical simulator for a quantum-circuit domain-specific language

Project members

Sasmit Ranjan	24BCI0160
Xitiz Rana	24BAI0120
Naveen Kishore M	24BKT0160
Pranesh M	24BKT0143

1. Introduction

QCompile is a C++ compiler for a compact domain-specific language (DSL) used to describe quantum circuits. A program contains named qubits, gate operations, optional numeric expressions for RZ rotations, and measurement statements. QCompile translates this source into a simple instruction list and executes that list on a classical complex-valued state-vector simulator. It is intended for small, inspectable examples and does not communicate with real quantum hardware.

The project applies standard compiler construction ideas to a quantum-circuit setting. Its driver, src/main.cpp, connects lexical analysis, parsing, semantic checking, code generation, and simulation. The command-line program prints intermediate representations, so the translation can be followed from source text to the final probabilities and sampled result.

2. Objectives

- Define a small readable circuit language with qubit declarations, H, X, Z, CNOT, RZ, measurement, variables, and arithmetic expressions.
- Implement a complete lexer-to-simulator pipeline with line-numbered error reporting.
- Represent source statements and expressions in an AST and validate rules that token shape alone cannot establish.
- Generate executable numeric instructions and demonstrate circuit behaviour using a classical state-vector simulation.

3. System / Compiler Architecture

The pipeline is sequential. A lexer or parser error prevents later stages from running. If semantic checking collects errors, code generation and simulation are also skipped. The implemented stages are summarized below.

Stage	Implementation in QCompile
Lexical analysis	src/lexer.cpp scans characters into Token values, preserving text, line, and column. It recognizes keywords, gates, identifiers, numbers, punctuation, operators, newlines, and EOF.
Parsing	src/parser.cpp uses recursive-descent functions to build a Program/Statement AST. parseExpression, parseTerm, and parseFactor implement arithmetic precedence, unary minus, and parentheses.
Semantic checking	src/semantic_checker.cpp walks statements in source order, checks declarations, arity, CNOT operands, parameters, undefined variables, and division by zero, and evaluates numeric expressions.
Code generation	src/code_generator.cpp skips declarations and emits a flat vector of Instruction objects. Qubit names are changed to zero-based indices; RZ parameters are stored as doubles.
Simulation	src/simulator.cpp stores amplitudes, applies gates/CNOT, and samples measurement.

4. Implementation

The shared data model is defined in include/ast.h. A Program stores statements in source order. A Statement represents a qubit declaration, variable declaration, or gate operation. Expression nodes represent number literals, identifiers, binary operations, and unary minus. Vectors are used for child expressions and lists of qubits rather than an inheritance hierarchy.

The lexer in src/lexer.cpp scans the complete source string and emits Token objects. It distinguishes the keywords qubit, let, and measure from the supported gate names H, X, Z, CNOT, and RZ. It also records source locations. The parser in src/parser.cpp consumes the token vector with peek, advance, match, and expect helpers. Its three expression functions give multiplication and division higher precedence than addition and subtraction; parentheses and unary minus are handled in parseFactor.

Semantic checking is performed after parsing. The checker keeps declaration-order vectors for qubits and variables. It verifies duplicate declarations, undeclared operands, gate arity, different CNOT control and target qubits, the RZ parameter rule, undefined variables, and division by zero. Valid expressions are evaluated to double values, because the simulator needs concrete numeric angles.

Code generation converts each non-declaration statement into an Instruction. It preserves operation order and maps names to zero-based indices. The simulator then uses a vector of complex<double> amplitudes. Single-qubit gates update amplitude pairs selected by a bit mask; CNOT swaps appropriate basis-state pairs. Measurement computes std::norm for each amplitude and samples with std::mt19937 seeded with 12345. The current measurement operation prints a result but does not collapse the state vector.

5. Workflow / Execution

- Read a source file such as samples/bell_state.txt and append line separators for the lexer.
- Tokenize the text and build an AST containing the declarations and operations.
- Check declarations, operands, arity, and expressions in source order. Stop before execution if errors are found.
- Generate APPLY and MEASURE instructions with integer qubit indices.
- Run the instruction list, print non-negligible probabilities, and sample the requested measurement bits.

6. Screenshots / Implementation Evidence

The following captures are from the project folder. They show the executable printing intermediate compiler stages and final simulator output rather than only showing source code.

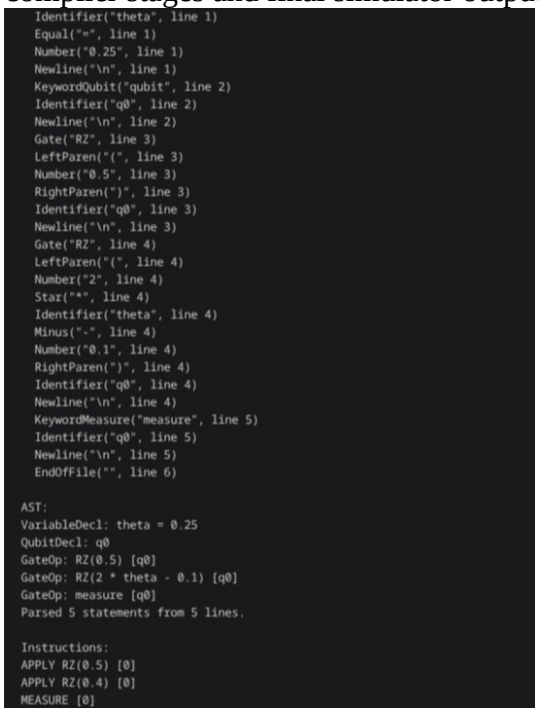


Figure 1: Bell-state compiler and simulator output.

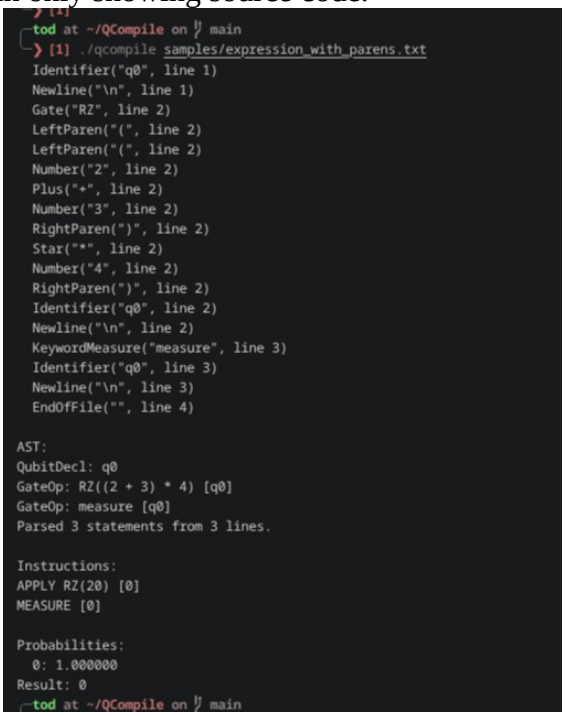


Figure 2: Parameterized-gate compiler output.

7. Results / Observations

The implemented project demonstrates a connected five-stage pipeline. For the Bell-state sample, the lexer identifies the declarations and operations, the parser prints a four-statement AST, and the generator emits APPLY H [0], APPLY CNOT [0, 1], and MEASURE [0, 1]. The simulator displays probabilities of 0.500000 for 00 and 11 and, with the documented seed, prints the sample result 11. This is an observed result of the supplied implementation; it is not a performance measurement.

The parameterized sample demonstrates that variables and expressions are not only accepted syntactically. let theta = 0.25 and RZ(2 * theta - 0.1) are evaluated by the semantic checker, and the generated instruction contains RZ(0.4). The sample set also includes parser and semantic-error cases. For example, using theta before its declaration produces an undefined-variable error and prevents instruction generation.

8. Scope and Limitations

- The simulator is a straightforward state-vector implementation whose state size grows as 2^n, so it is intended for small circuits.
- The language has no control flow, registers, subroutines, or circuit optimization; source operation order is preserved.
- Measurement samples and displays an outcome but does not update the state vector after measurement.
- The implementation targets simulation on an ordinary computer, not execution on quantum hardware.

9. Conclusion

QCompile achieves the project goal of implementing a small but complete compiler for a quantum-circuit DSL. Its lexer, recursive-descent parser, semantic checker, instruction generator, and state-vector simulator work together and expose their intermediate results. The project therefore provides a practical demonstration of compiler stages while keeping the language and simulator small enough to understand and run locally.

10. References

- QCompile project README.md — language overview, build instructions, samples, and pipeline summary.
- QCompile Technical Documentation, DOCUMENTATION.md — grammar, source-file reference, pipeline trace, and design scope.
- Implementation files: include/ast.h, include/lexer.h, include/parser.h, include/semantic_checker.h, include/code_generator.h, include/simulator.h, and corresponding files in src/.
- Github repository - https://github.com/TodScissorhands/QCompile